

WEEK 19.3 ASSIGNMENT

NAME: MOHAMMED FAISAL QURESHI

H.TNO: 2303A53015

BATCH: 46

Task 1 – Python to C++ Conversion

Task:

Translate a simple Python class into C++ using AI assistance.

Instructions:

- Prompt AI to convert a Python class representing a Student with attributes and methods into equivalent C++ code
- Ensure proper handling of:
 - Constructors
 - Data types
 - Access specifiers
- Compile and run both versions to verify output consistency

Expected Output:

An equivalent working C++ class translated from Python

PROMPT:

Convert a Python class named Student with attributes (name, age, marks) and a method display() into equivalent C++ code with proper constructor and access specifiers.

CODE:

class Student:

```
def __init__(self, name, age, marks):
```

```
    self.name = name
```

```
    self.age = age
```

```
    self.marks = marks
```

```
def display(self):
```

```
    print("Name:", self.name)
```

```
    print("Age:", self.age)
```

```
    print("Marks:", self.marks)
```

```
s = Student("Ali", 20, 85)
```

```
s.display()
```

C++ TRANSLATED CODE:

```
#include <iostream>
```

```
using namespace std;
```

```
class Student {  
private:  
    string name;  
    int age;  
    int marks;  
public:  
    Student(string n, int a, int m) {  
        name = n;  
        age = a;  
        marks = m;  
    }  
    void display() {  
        cout << "Name: " << name << endl;  
        cout << "Age: " << age << endl;  
        cout << "Marks: " << marks << endl;  
    }  
};  
int main() {  
    Student s("Ali", 20, 85);  
    s.display();  
    return 0;  
}
```

OUTPUT:

Name: Ali

Age: 20

Marks: 85

Task 2 – Java to Python Function Conversion

Task:

Convert a Java method that checks whether a number is prime into Python.

Instructions:

- Ask AI to translate a Java isPrime() method into Python
- Test the function with multiple inputs
- Ensure the Python version follows Pythonic syntax and logic

Expected Output:

A correct Python function equivalent to the original Java prime-checking method.

PROMPT:

Convert a Java method isPrime() into Python.

CODE:

```
public static boolean isPrime(int n) {  
    if (n <= 1) return false;  
    for (int i = 2; i <= n/2; i++) {  
        if (n % i == 0)  
            return false;  
    }  
    return true;  
}
```

TRANSLATED TO PYTHON:

```
def is_prime(n):  
    if n <= 1:  
        return False  
    for i in range(2, n//2 + 1):  
        if n % i == 0:  
            return False  
    return True
```

```
print(is_prime(7))  
print(is_prime(10))
```

OUTPUT:

```
True  
False
```

Task 3 – Pseudocode to Python Implementation

Task:

Translate a given pseudocode algorithm into Python using AI.

Instructions:

- Provide AI with pseudocode for sorting numbers (e.g., bubble sort or selection sort)
- Ask AI to convert it into executable Python code
- Validate correctness using sample input lists

Expected Output:

A working Python program that correctly implements the pseudocode logic.

PROMPT:

Convert bubble sort pseudocode into Python.

CODE:

```
FOR i from 0 to n-1
  FOR j from 0 to n-i-1
    IF arr[j] > arr[j+1]
      SWAP arr[j], arr[j+1]
```

TRANSLATED TO PYTHON:

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
```

```
arr = [5, 2, 9, 1]
bubble_sort(arr)
print(arr)
```

OUTPUT:

[1, 2, 5, 9]

Task 4 – SQL to Pandas Query

Task:

Translate a **SQL query** into a **Pandas DataFrame operation** in Python.

Instructions:

- Provide AI with a SQL query (e.g., SELECT name, salary FROM employees WHERE salary > 50000).
- Ask AI to generate equivalent Pandas code.
- Test the generated code on a sample DataFrame.

Expected Output:

- Equivalent Pandas query that retrieves the correct subset of data.

PROMPT:

Convert SQL query SELECT name, salary FROM employees WHERE salary > 50000 into Pandas.

CODE:

```
SELECT name, salary FROM employees WHERE salary > 50000;
```

PYTHON PANDA:

```
import pandas as pd
```

```
data = {  
    'name': ['Ali', 'John', 'Sara'],  
    'salary': [60000, 40000, 70000]  
}
```

```
df = pd.DataFrame(data)
```

```
result = df[df['salary'] > 50000][['name', 'salary']]  
print(result)
```

OUTPUT:

```
   name  salary  
0  Ali   60000  
2  Sara  70000
```

Task 5 – Real-Time Application: Algorithm Translation Across Languages

Scenario:

A company maintains algorithms written in different programming languages.

Examples:

- Converting a Python searching algorithm into C
- Translating a Java sorting program into JavaScript

Instructions:

- Use AI to translate a selected algorithm between two programming languages
- Execute and test both versions
- Document translation challenges such as:
 - Syntax differences
 - Library support
 - Memory management

Expected Output:

Equivalent, tested code implementations in two different programming languages.

PROMPT:

Convert Python linear search algorithm into C.

CODE:

```
def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i
    return -1
print(linear_search([10, 20, 30], 20))
```

TRANSLATED TO C:

```
#include <stdio.h>
int linear_search(int arr[], int n, int key) {
    for(int i = 0; i < n; i++) {
        if(arr[i] == key)
            return i;
    }
    return -1;
}
int main() {
    int arr[] = {10, 20, 30};
    int result = linear_search(arr, 3, 20);
    printf("%d", result);
    return 0;
}
```

OUTPUT:

1

